임베디드 시스템 설계 – Line Tracer Project

(기여도 모두 동일)

Main Idea

차선 인식 - (양쪽차선/한쪽차선/탈선/정지선)

1) Line scan Camera 의 밝기 값은 조명의 세기, 빛의 반사 등에 따라 민감하게 변하므로 고정된 임계값을 이용하면 오작동하기 쉽다. 따라서 매 프레임마다 유효구간(5 ~ 122)에서 가장 어두운 값과 가장 밝은 값의 평균을 Threshold 로 갱신하여 현재 프레임의 상대적인 대비를 기준으로 검정/흰색을 분리한다.

2) 양 쪽 차선을 인식한 경우, 좌/우 라인의 위치로 Center_ind 를 계산한다. 왼쪽 영역(5 ~ 63), 오른쪽 영역(64 ~ 122)에서 camera_data[i] < threshold 인 위치를 탐색하여 중앙점을 찾는다.

3) 만약 한 쪽 차선만 인식한 경우, center 가 갑자기 극단적인 값을 가져 조향이 튀며 탈선 할 수 있으므로 Center_ind 를 1 씩만 이동시키면서 완만하게 보정하여 조향이 급변하지 않도록 안정화한다.

4) 정지선은 화면 전체에 검정색 픽셀이 많이 깔리는 패턴이다. 따라서 Threshold 보다 작은 픽셀 개수 blackCount 로 정의하여 처리한다. 가장 중요한 부분은 중복 카운트를 방지하는 것이다. 정지선은 출발할 때, 혹은 지나칠 때 여러 프레임 동안 계속 보이므로, 한 번 감지하면 stopline_no_detect_count 를 이용하여 약 몇 초동안은 다시 중복 검출하지 않도록 한다. 이렇게 해서 stopline_hits_number 가 처음 출발 포함 3 회가 되면 즉시 정지하도록 한다.

5) 차선을 인식하지 못한 경우 (혹은 탈선), 양 쪽 차선 모두를 놓친 상황에서는 잘못된 center 로 계속 주행하면 바로 차선 이탈/복귀 불가능으로 이어진다. 이를 막기 위해 프레임 대비가 거의 없는 경우를 차선 미인식으로 정의하여 (no_2line = true)복구 모드로 전환하고 저속 후진하여 다시 line 을 찾기 시작한다. 이때, stopline_detected = false 로 정지선 감지 상태 또한 강제로 off 시킨다.

모터 제어 - (직선/곡선)

1) Loop(200Hz)에서 b_left(좌측 속도), b_right(우측 속도)를 계산하여 갱신하고, Timer1 ISR(20kHz)은 DDA 로 그 값에 맞는 STEP 펄스를 생성한다. 따라서 제어 구조는 Center 기반으로 속도와 조향을 결정하여 b_left/right 를 갱신하고, 그를 실제 펄스로 변환한다.

2) 직선/곡선 판정은 중심으로부터의 오차 err 과 오차 변화량 derr 을 함께 사용하여 구분하였다. Err 과 derr 의 임의의 두 임계값보다 동시에 작으면 직선, err 혹은 derr 의 임의의 임계값보다 하나라도 큰 경우 코너에 진입했다고 판단하며, 다음과 같이 속도를 조절한다.

   코너: current_speed -= 2000 (최소 = base_speed) 직선:
   current_speed + = accel_rate (최대 = top_speed) 그 외:
   current_speed -= 25 (최소 = base_speed)

   추가적으로 직선 주행에서는 중심이 거의 중앙(63 ~ 64)이면 b_left = b_right = v(current_speed)로 불필요한 조향을 없애고 직선에서는 안정적으로 가속하도록 한다.

3) err 의 크기에 따라 current_k 를 바꾸는 방식으로 곡선 구간에서의 조향 민감도를 크게 만든다.

   abs_err < 8 이면 직선에 가까우므로 current_k = 15 로 부드러운 조향, 그 외는 current_k = 110 으로 강한 조향을 준다.

4) 곡선 구간에서 조향을 더 세밀하게 제어하기 위해 바깥 바퀴와 안쪽 바퀴의 속도를 조절한다.

   바깥 바퀴 out 은 current_speed 로 고정하고, 안쪽 바퀴 in 은 current_k * err 만큼 current_speed 에서 감속한다. 이때, out_wheel 와 in_wheel 의 최소속도를 설정해두어 너무 저속으로 떨어지는 것 또한 방지한다.

   전체적인 주행 구간 중 좌회전이 많으므로 좌회전의 경우 비대칭 보정을 추가해 빠른 복귀가 가능하도록 해주었다. 즉, 바깥 바퀴는 더 빠르게, 안쪽 바퀴는 더 느리게 하여 속도차를 키우고, 빠르게 복귀할 수 있도록 하였다.

시행착오 과정

1) 고정 임계값 기반 라인 인식의 한계

초기에는 차선과 바닥을 구분하기 위해 고정된 임계값을 사용하였다. 실제 주행 환경에서는 조명의 세기, 빛 반사 등에 의해 차선 인식의 결과가 매번 달라지는 문제가 발생하였고, 차선을 놓치는 현상으로도 이어졌다.

이를 해결하기 위해 매 프레임마다 유효 구간에서 최소 밝기 값, 최대 밝기 값을 추출하여 그 평균을 임계값으로 설정하는 조금 더 robust 한 방법을 적용하였고, 트랙 전 구간에서 라인 인식이 안정적으로 유지되는 효과를 얻었다.

2) 고속 주행 시 코너 구간에서의 이탈 문제

초기에는 중심 오차 err 만을 기준으로 속도를 조절하였고, 이 방식은 코너 진입 상태를 너무 늦게 적용하여 코너에서 감속이 늦어지고, 그대로 차선을 이탈하는 문제로 이어졌다.

이를 해결하기 위해 중심 오차 err 뿐만 아니라 오차 변화량 derr 을 함께 이용하여 코너 진입을 더 세밀하게 판단하였다. 기존 판단 조건 err 과 함께 derr(=차량이 얼마나 빠르게 (계산된)중심점으로부터 벗어나고 있는지), 즉 회전이 시작되는 순간을 더 민감하게 판단하게 된다.

이 문제는 “차량의 속도”와 “회전 구간 통과” 그리고 “다른 구간에서 나타나는 부작용”의 Trade-Off 가 발생하여 매우 많은 튜닝 과정이 있었다. 가장 기본적으로 랩 타임을 줄이기 위해 속도를 높이면 회전 구간에서 여전히 이탈하였다. 이러한 이유로, 고속에서 회전 구간을 잘 인식하도록 derr 조건을 느슨하게 주면 직선 구간에서의 비틀거림과 연속적 회전 구간에서의 미끄러짐이 발생하였다.

다양한 환경(다른 조의 맵, 조명의 세기가 다른 위치)에서 시도해보며 최적의 값을 찾아냈다.

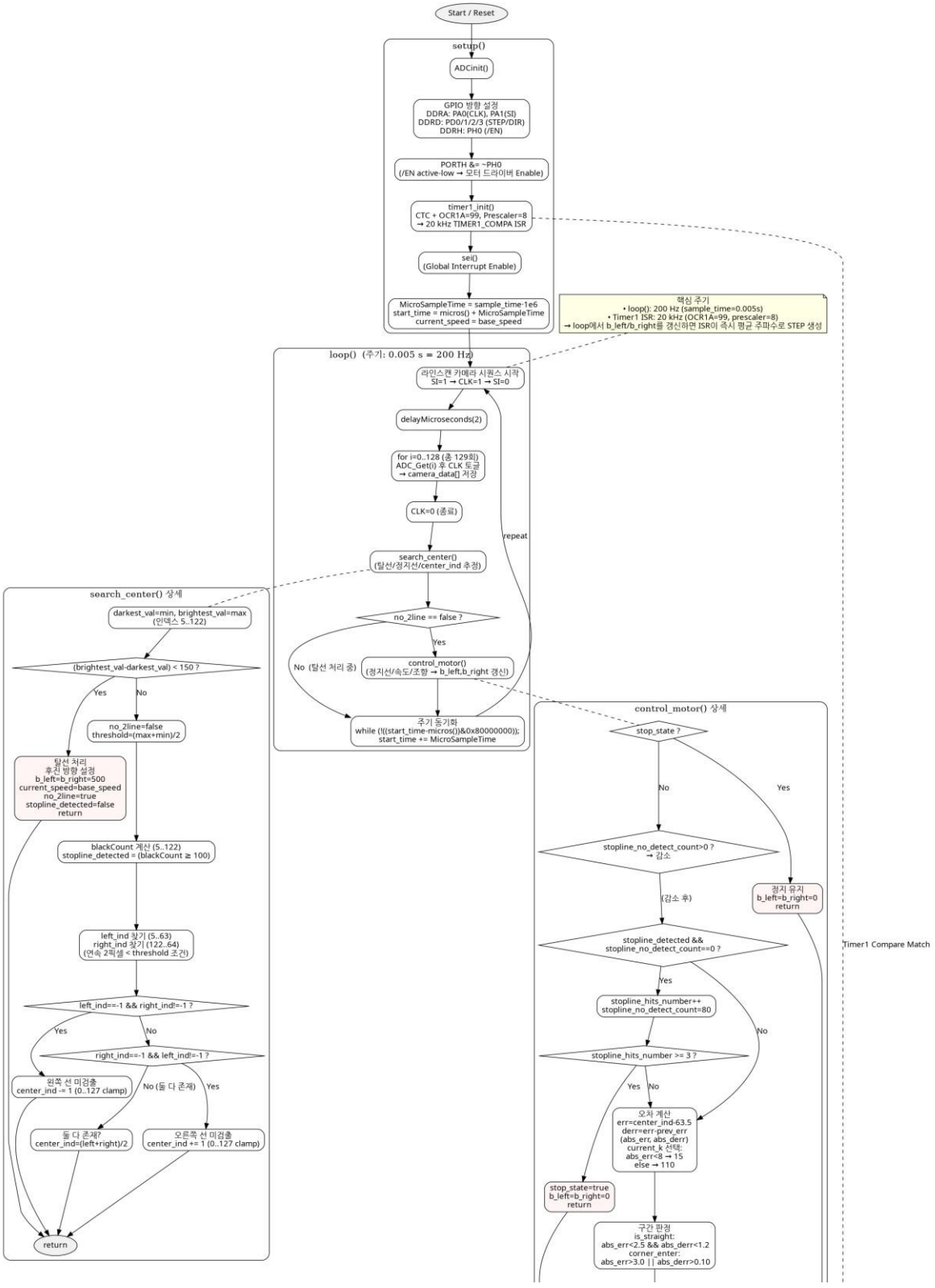
3) 정지선 중복 인식으로 인한 랩 카운트 문제

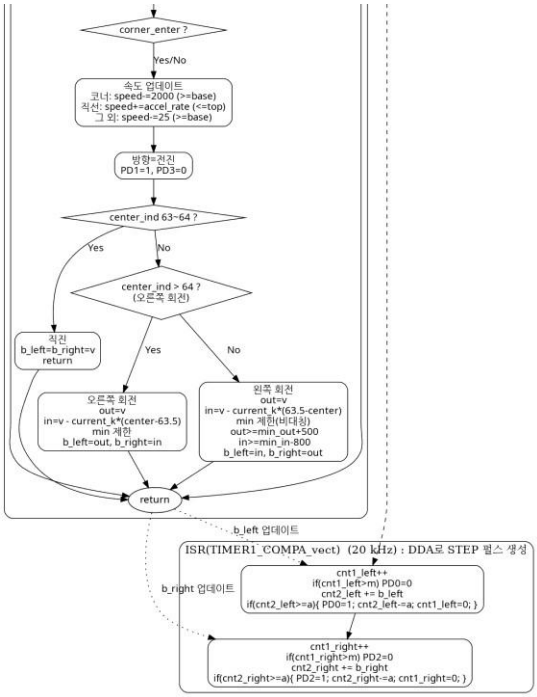
정지선이 여러 프레임에 걸쳐 인식되기 때문에 한 번 지나갈 때 랩 카운트가 여러 번 증가하는 문제가 발생하였다. (혹은 다른 곳에서 잘못 정지선으로 인식하는 경우도 포함)

이를 해결하기 위해 Serial.print 로 디버깅을 하며 stopline_no_detect 와 같은 일정 프레임 동안의 재검출을 방지하는 카운트를 도입하였고, 엉뚱한 구간에서 정지선으로 인식하는 경우를 방지하기 위해 stopline_blackcount_th 정지선 인식 임계값을 더 강하게 줘서 해결하였다.

4) 햇빛에 의한 광량 증가로 차선 인식 불가능 문제

강한 햇빛에 의해서 광량이 크게 증가하면, 검은 차선도 max 값으로 인식하여 아예 차선 인식이 안 되는 문제가 발생했고 카메라의 저항을 키워서 bias 를 낮추어 해결하였다.


Start / Reset
setup()
ADCinit()
GPIO 방향 설정
DDRA: PA0(CLK), PA1(SI)
DDRD: PD0/1/2/3 (STEP/DIR)
DDRH: PH0 (/EN)
PORTH &= ~PH0
(/EN active-low → 모터 드라이버 Enable)
timer1_init()
CTC + OCR1A=99, Prescaler=8
→ 20 kHz TIMER1_COMPA ISR
sei()
(Global Interrupt Enable)
MicroSampleTime = sample_time·1e6
start_time = micros() + MicroSampleTime
current_speed = base_speed
핵심 주기
• loop(): 200 Hz (sample_time=0.005s)
• Timer1 ISR: 20 kHz (OCR1A=99, prescaler=8)
→ loop에서 b_left/b_right를 갱신하면 ISR이 즉시 평균 주파수로 STEP 생성
loop() (주기: 0.005 s = 200 Hz)
라인스캔 카메라 시퀀스 시작
SI=1 → CLK=1 → SI=0
delayMicroseconds(2)
for i=0..128 (총 129회)
ADC_Get(i) 후 CLK 토글
→ camera_data[] 저장
CLK=0 (종료)
search_center()
(탈선/정지선/center_ind 추정)
no_2line == false ?
Yes
No (탈선 처리 중)
control_motor()
(정지선/속도/조향 → b_left,b_right 갱신)
주기 동기화
while (!((start_time-micros())&0x80000000));
start_time += MicroSampleTime
repeat
search_center() 상세
darkest_val=min, brightest_val=max
(인덱스 5..122)
(brightest_val-darkest_val) < 150 ?
Yes
No
no_2line=false
threshold=(max+min)/2
탈선 처리
후진 방향 설정
b_left=b_right=500
current_speed=base_speed
no_2line=true
stopline_detected=false
return
blackCount 계산 (5..122)
stopline_detected = (blackCount ≥ 100)
left_ind 찾기 (5..63)
right_ind 찾기 (122..64)
(연속 2픽셀 < threshold 조건)
left_ind==-1 && right_ind!=-1 ?
Yes
No
right_ind==-1 && left_ind!=-1 ?
왼쪽 선 미검출
center_ind -= 1 (0..127 clamp)
No (둘 다 존재)
Yes
둘 다 존재?
center_ind=(left+right)/2
오른쪽 선 미검출
center_ind += 1 (0..127 clamp)
return
control_motor() 상세
stop_state ?
No
Yes
stopline_no_detect_count>0 ?
→ 감소
(감소 후)
정지 유지
b_left=b_right=0
return
stopline_detected &&
stopline_no_detect_count==0 ?
Yes
No
stopline_hits_number++
stopline_no_detect_count=80
stopline_hits_number >= 3 ?
Yes
No
오차 계산
err=center_ind-63.5
derr=err-prev_err
(abs_err, abs_derr)
current_k 선택:
abs_err<8 → 15
else → 110
stop_state=true
b_left=b_right=0
return
구간 판정
is_straight:
abs_err<2.5 && abs_derr<1.2
corner_enter:
abs_err>3.0 || abs_derr>0.10
Timer1 Compare Match


corner_enter ?
Yes/No
속도 업데이트
코너: speed=2000 (>=base)
직선: speed+=accel_rate (<=top)
그 외: speed-=25 (>=base)
방향=전진
PD1=1, PD3=0
center_ind 63~64 ?
Yes
No
center_ind > 64 ?
(오른쪽 회전)
Yes
No
직진
b_left=b_right=v
return
오른쪽 회전
out=v
in=v - current_k*(center-63.5)
min 제한
b_left=out, b_right=in
왼쪽 회전
out=v
in=v - current_k*(63.5-center)
min 제한(비대칭)
out>=min_out+500
in>=min_in-800
b_left=in, b_right=out
return
b_left 업데이트
ISR(TIMER1_COMPA_vect) (20 kHz) : DDA로 STEP 펄스 생성
b_right 업데이트
cnt1_left++
if(cnt1_left>m) PD0=0
cnt2_left += b_left
if(cnt2_left>=a){ PD0=1; cnt2_left-=a; cnt1_left=0; }
cnt1_right++
if(cnt1_right>m) PD2=0
cnt2_right += b_right
if(cnt2_right>=a){ PD2=1; cnt2_right-=a; cnt1_right=0; }